

Proof catalog: every theorem, how it's proved, and the lemmas it leans on

Hoang Nguyen

Abstract

Every named theorem proved in `Gc/Model/` and `Gc/Reachability/Reachable/`, listed with a one-line statement, a rough sketch of the proof, and the key supporting lemmas it actually calls. `Gc/Reachability/Referencable/` is excluded (deprecated, superseded by `Reachable/`; see `CLAUDE.md`). Definitions and formal statements already given in full in `writeup/main.tex` are cited by name, not restated.

Contents

1	Notation	2
2	<i>Gc/Model: ValidConfig</i> preservation	2
2.1	<code>RuntimeConfig.start_valid</code>	2
2.2	<code>enter_valid</code> (<code>enter xf bridgeVar</code>)	2
2.3	<code>exit_valid</code> (<code>exit</code>)	3
2.4	<code>fieldAsgn_valid</code> (<code>fieldAsgn xf y</code>)	3
2.5	<code>makeObjRegion_valid</code> (<code>makeObjRegion x</code>)	4
2.6	<code>makeObjStack_valid</code> (<code>makeObjStack x</code>)	4
2.7	<code>makeRegion_valid</code> (<code>makeRegion x</code>)	5
2.8	<code>merge_valid</code> (<code>merge x</code>)	5
2.9	<code>varAsgn_valid</code> (<code>varAsgn x yf</code>)	6
2.10	<code>swap_valid</code> (<code>swap x yf</code>)	6
2.11	<code>allPreserve_ValidConfig</code>	7
3	<i>Gc/Reachability/Reachable: semantics and the confinement toolkit</i>	7
3.1	Semantics-level theorems (<code>Semantics.lean</code>)	7
3.2	The confinement toolkit (<code>Lemmas/Common.lean</code>)	7
3.3	<i>CR6</i> (<code>CR6.lean</code>)	8
4	<i>FrameReachableAtLaterFrame</i>	9
4.1	<code>RuntimeConfig.start_frameReachableAtLaterFrame</code>	9
4.2	The nine per-operation proofs	9
4.3	<code>frameReachableAtLaterFrame_step_all</code>	10
5	<i>StackReachable_invariant_for_suspended_region_objects</i>	11
5.1	<code>stackReachable_invariant_all</code>	11
6	What's not proved yet	11

1 Notation

$\langle S, H \rangle$: stack of frames, heap of regions. $\text{Rld}(rid)/\text{Old}(oid)$: references. $a.\text{loc}(\langle S, H \rangle)$: where a reference resolves ($\text{Stk}(fid)$, $\text{Rgn}(rid)$, or undefined). $a.\text{objAt}(\langle S, H \rangle)$: the object an **Old** names. The ten invariants $L1, L2, H1, H2, H3, S1, S2, S3, HS1, HS2$ making up *ValidConfig* are defined in `main.tex`; one-line reminders below. $HS1/HS2$ are worth flagging: they don't follow from the other eight, since $H2/H3/S2/S3$ all presuppose a reference that already resolves somewhere and say nothing about one that resolves to nothing, so without $HS1/HS2$ a stale reference could sit inert and later collide, uncaught, with a freshly allocated id.

Invariant	What it rules out
$L1$	the same object id allocated twice
$L2$	a frame executing inside a <i>Closed</i> region
$H1$	a region whose bridge object isn't in its own <i>objMap</i>
$H2$	two live $\text{Rld}(rid)$ references to the same region
$H3$	a region's contents pointing outside that region
$S1$	two stack frames sharing a region
$S2$	a frame referencing a stack object owned by a later frame
$S3$	a frame referencing a region owned only by a later frame
$HS1$	a dangling Old
$HS2$	a dangling Rld

2 Gc/Model: *ValidConfig* preservation

Each `<op>_valid` theorem has the shape $\text{ValidConfig}(cfg) \rightarrow op\ cfg = \text{some } cfg' \rightarrow \text{ValidConfig}(cfg')$, proved by destructing the operation via its `<op>_cases` lemma and discharging the ten invariants one at a time. Below, each invariant is marked either trivial (with the one-line reason) or given its key lemma.

2.1 `RuntimeConfig.start_valid`

Base case: one root region (Open, bridge 0), one root frame executing in it, with empty *objMap/varMap* everywhere. All ten invariants reduce, after unfolding *RuntimeConfig.start*'s definition, to a fact about a one- or two-element list — e.g. $L1$ is exactly `List.nodup_singleton` on the singleton object-id list `[0]`, and $S2/S3$ are vacuous since the one frame's *refs* list is empty — closed by `decide/simp`. No induction, no reused corollaries: this file only exists to anchor the induction that follows.

2.2 `enter_valid (enter xf bridgeVar)`

Pushes a new, empty frame into region `rid'` (found via `resolveFA xf`, must be **Closed**), flips `rid'` to **Open**.

L1 trivial: no *objMap* touched (new frame is empty; heap change is a status flip).

L2 the new frame's region is **Open** by construction (that's the whole point of `enter`); other frames' `heap.lookup` calls survive the update via `AList.lookup_insert_ne`, since the entered region key `rid'` is distinct from every other on-stack frame's own region (by $S1$, the new frame's own added at the very end can't collide with anyone else's).

- H1** trivial: the status flip doesn't touch *bridgeObjectId/objMap*, so whatever object was already the bridge stays the bridge.
- H2** trivial: no reference created, removed, or relocated; the bridge object was already counted, and it stays the region's own bridge.
- H3** **enter_corollary_2**: *.loc(.)*'s definition never reads *region.status*, only *objMap* membership, so it's unconditionally unaffected by a pure status flip and every existing *H3* witness transfers as-is.
- S1** **enter_corollary_1** (region-uniqueness from *L1*) plus *L2*'s contrapositive: a Closed region (what **enter** requires) can't already be owned by an on-stack frame, so the new frame's region is distinct from every existing one, and appending it at the end can't create a collision with itself either.
- S2, S3** the new frame is appended with empty *objMap/varMap*, so there is nothing new to check for it; existing frames keep the same index and the same content, so their own bounds are exactly the pre-mutation ones via **enter_corollary_2**.
- HS1, HS2** trivial: nothing new enters *refs*.

2.3 exit_valid (exit)

Pops the last frame, flips its region back to Closed.

- L1** trivial: the popped frame's ids are simply dropped, and by *L1* on *cfg* they were already disjoint from everything remaining.
- L2** surviving frames' own **heap.lookup** results are untouched by the status flip, except each survivor needs its region to be $\neq$ the popped one so the flip doesn't accidentally invalidate it too; that's exactly *S1* (regionIds pairwise distinct across the stack), applied via **exit_regionId_ne**.
- H1** trivial: status flip only, *bridgeObjectId/objMap* untouched.
- H2** trivial: no reference created, removed, or relocated by a status flip.
- H3** the popped frame's own contents vanish with it, and by *L1* on the pre-mutation config its object ids were already disjoint from every surviving frame's and every region's, so nothing else could have depended on locating through them; survivors' own *H3* witnesses are otherwise untouched.
- S1, S2, S3** index bookkeeping over the shrunk stack: every surviving frame keeps the same position it had before (the popped frame was the *last* one, so nothing shifts), and nothing in a survivor's *refs* could have pointed at the now-gone last slot without already violating *S2/S3* pre-mutation. Proved by **exit_S1, exit_S2, exit_S3**.
- HS1, HS2** the popped frame's *refs* disappear along with it, shrinking *cfg.refs*; nothing new is added, so both invariants persist on the smaller set.

2.4 fieldAsgn_valid (fieldAsgn xf y)

Overwrites one field of an object already resolved via **resolveV**, either on the stack (same frame) or inside the writer's own Open region (target must resolve into that same region too).

- L1, L2, S1, H1** trivial: no id allocated, no frame/region/bridge changed, only a field value inside an already-present object.
- H2** the new value written into the field is always an Old (this rule never stores an Rld), while the field *might* have held an Rld before the overwrite, so the Rld-count can only go down or stay the same, never up: **fieldAsgn_corollary_object_insert_count_le** states this at the

single-object level, lifted to the whole config through `_objMap_insert_bind_refs_count_le/_heap_insert_refs_count_le`.

- H3** the STACK branch doesn't touch *H3* at all (it's a heap-only invariant); the REGION branch's own side condition (the new value must resolve into the *same* region as the write target) is exactly *H3*'s content, formalized via `fieldAsgn_corollary_region_loc_eq/_region_heap_objectIds_perm`, which also re-derive that every *other* object's location survives the heap-entry reorder caused by inserting at the (already-present) region key.
- S2, S3** both `xf.root` (the container being written into) and `y` (the new value) were already successfully resolved via `resolveV` *before* the mutation, so they were already index-bounded by *S2/S3* on the old config; `fieldAsgn_corollary_resolveV_loc_ancestor` carries that pre-existing bound over to the newly-stored occurrence, so nothing new needs to be independently justified.
- HS1** `fieldAsgn_corollary_resolveV_oid_mem_objectIds`: the written value was already live, so already counted in *objectIds*.
- HS2** trivial: no Rld ever passes through this rule.

2.5 makeObjRegion_valid (makeObjRegion x)

Allocates a fresh, empty object directly inside the writer's own Open region.

- L1** `RuntimeConfig.freshObjectId_not_mem`: the fresh id can't already be present.
- L2, S1** trivial: no frame/region status or ownership changes.
- H1** trivial: the pre-existing bridge id is untouched by inserting a new key.
- H2** `makeObjRegion_corollary_heap_refs_perm`: the new object is empty (no refs), so *Heap.refs* is an exact permutation of before.
- H3** the fresh object trivially satisfies it: it's inserted directly into the writer's own region's *objMap*, so it resolves right back into that same region by construction. Every *other*, pre-existing object's location needs `makeObjRegion_corollary_loc_eq`, since inserting at the (already-present) region key reorders the heap's `.entries` list even though it doesn't change any key set — that reorder is why `makeObjRegion_corollary_region_unique` (an *L1* corollary: an id lives in at most one region) is needed, to show the reordered search still lands on the same region as before.
- S2, S3** `makeObjRegion_corollary_loc_fresh`: the fresh object always resolves into the heap, never onto the stack (its *Frame.objMap* is untouched, only the region's), so *S2* (a stack-only invariant) is vacuously satisfied for it, and *S3* is trivial since its owning frame is exactly the frame that just created it.
- HS1** trivial: the fresh object is now in *objectIds* by construction.
- HS2** trivial: no Rld touched.

2.6 makeObjStack_valid (makeObjStack x)

Allocates a fresh, empty object directly on the current frame's own (stack-local) *objMap*; the heap is untouched entirely (`makeObjStack_corollary_1`).

- L1** `freshObjectId_not_mem`, on the stack side.
- L2, H1, H2** trivial: heap untouched.
- H3** doesn't apply to the fresh object (it resolves to *Stk*, not *Rgn*); heap side is untouched for everything else.
- S1** trivial.
- S2, S3** the fresh object is inserted directly into the current (last) frame's own *objMap*, so it resolves

to that frame’s own index, trivially $\leq$ itself; every other object’s location is unaffected since the insert happens at a fresh key (no reorder at all, unlike `makeObjRegion`’s heap-side insert at an existing key).

HS1, HS2 trivial: the fresh object is empty, and nothing points at it yet except the one new var binding, which *HS1* is about the fresh id itself being in *objectIds* (true by construction), not about anything pointing at it.

2.7 `makeRegion_valid (makeRegion x)`

Allocates a fresh Closed region with its own fresh bridge object, storing `Rld` to it in the current frame’s variable `x`.

L1 `freshObjectId_not_mem` for the new bridge object.

L2 trivial for existing frames (insert at a genuinely fresh heap key disturbs nothing); nobody executes in the new region yet.

H1 trivial by construction: the fresh region’s only object is its own bridge.

H2 the new `Rld(freshRegionId)` is stored exactly once, in `x`; it starts at count 0 because a not-yet-existing region id can’t already have been referenced (*HS2* on the old config says every stored `Rld` already resolves into *heap.keys*, and `freshRegionId_not_mem` says the fresh id isn’t in *heap.keys* yet), so adding the one new occurrence brings the count to exactly 1, not over the bound.

H3 trivial for the fresh region; `makeRegion_corollary_loc_eq` for everything else (fresh-key insert, no reorder needed).

S1 trivial: no existing frame’s `regionId` changes, and the new region isn’t any frame’s.

S2, S3 the fresh bridge object resolves into the heap (never the stack); its owning frame is the one that just created it.

HS1 trivial: fresh bridge object now in *objectIds*.

HS2 trivial: fresh region id now in *heap.keys* by construction.

2.8 `merge_valid (merge x)`

Folds a Closed region `rid'` into the current Open region `rid`: combines their *objMaps*, deletes `rid'`, rebinds `x` to `rid'`’s old bridge object. The busiest file (~1200 lines), the only operation deleting a heap key.

The load-bearing fact the two regions’ object-id sets are disjoint: `merge_corollary_rid_ne_regionId` (Open $\neq$ Closed) plus `merge_corollary_region_unique` (an *L1* corollary: no id in two heap entries). This alone rules out the union silently colliding or dropping an object, since `AList.union` in general erases duplicate keys from one side.

L1 `merge_corollary_union_append` (disjoint keys $\Rightarrow$ union is a plain append) plus `merge_corollary_heap_entries_perm` (the post-merge heap is a permutation of the merged pair plus everything untouched, via two nested `List.exists_of_kerase` extractions): *L1* then follows from permutation-invariance of `Nodup`.

L2, S1 trivial: only the last frame’s *varMap* and the two regions’ heap entries change.

H1 `rid`’s own bridge object (unchanged by the merge) is still present, since the union only ever adds keys from `rid'`’s side, never removes any of `rid`’s; `rid'`’s own bridge object is simply absorbed as an ordinary object into the merged *objMap* (its status as “the” bridge dies with the deleted region key, which is fine, since *H1* is only ever asked about heap entries that still exist).

H2 the deleted reference `Rld(rid')` simply disappears from *refs* entirely (both the region key and `x`’s

old value are gone), so its count drops to 0; the new `Old(region'.bridgeObjectId)` written to `x` was already counted once under `H1` of `rid`' pre-merge, and merging doesn't add a second occurrence.

- H3** `merge_corollary_loc_of_mem`: an object that used to live in *either* original region still resolves into the merged `rid` afterward, since the stack side of `.loc(·)`'s search is untouched (only `x`'s `varMap` entry changes, never any frame's `objMap`) and the heap side's uniqueness (from the disjointness fact above) pins `rid`'s merged entry down as the only heap entry whose `objMap` could contain that id.
- S2, S3** carried by the same `merge_corollary_loc_of_mem/index-split` machinery as **H3**: every object's location resolves to the same place (or the merged `rid`) as before, so the index bound it already satisfied under the old config still applies unchanged; `merge_corollary_stackWithIndex_split` handles the shared "last frame changed, rest didn't" bookkeeping that both invariants' proofs need.
- HS1, HS2** follow from the permutation facts above: nothing created or destroyed, only relocated.

2.9 varAsgn_valid (varAsgn x yf)

Two branches: rebind the frame's own bridge variable to an already-in-region value ("bridge branch"), or bind a fresh local variable to a resolved value ("fresh-var branch", requiring `resolveV x cfg = none`).

- L1** trivial both branches: no object created; bridge branch permutes `heap.objectIds` via a re-order (insert at an existing key), not a content change (`varAsgn_corollary_bridge_heap_objectIds_perm`).
- L2, S1** trivial: no region status or frame `regionId` touched.
- H1** bridge branch: the new bridge id is already inside the region by the rule's own side condition (`yfRef.loc(·) = Rgn(rid)`, `rid = frame.regionId`). Fresh-var: trivial.
- H2** `varAsgn_corollary_bridge_heap_refs_perm`: `bridgeObjectId` isn't read by `Region.refs` at all, so the bridge branch's heap refs are literally unchanged; fresh-var branch only relocates an existing reference into a new var slot.
- H3** `varAsgn_corollary_loc_eq`: `.loc(·)` is unconditionally preserved in both branches (literal equality for fresh-var; uniqueness-based, `varAsgn_corollary_region_unique`, for the bridge branch's heap reorder).
- S2, S3** bridge branch: trivial, since **S2/S3** are about `varMap/objMap` contents, and only the scalar `bridgeObjectId` changes. Fresh-var branch: `varAsgn_corollary_resolveV_loc_ancestor / _yfRef_ancestor` show the value being copied into the new var slot already had a valid owning ancestor frame on the stack *before* the copy (it was already reachable via `resolveV/resolveFA`), so writing it into one more var slot doesn't need a fresh justification.
- HS1** fresh-var branch: `varAsgn_corollary_yfRef_mem_refs` (the copied value was already live).
- HS2** trivial: no `Rld` is ever created or moved by `varAsgn`.

2.10 swap_valid (swap x yf)

The busiest rule syntactically (four branches: stack, region-object, region-region, region-bridge), each exchanging two live reference slots at once. Largest file (~2250 lines).

- L1, L2, S1** trivial: no allocation, only reassigning values at already-present keys.
- H1** the region-bridge branch rebinds `bridgeObjectId`; same argument as `varAsgn`'s bridge branch (new bridge already inside the region, by the rule's side condition).

H2 `swap_corollary_alist_insert_count_eq / _objMap_insert_bind_count_eq / _heap_insert_bind_count_eq`: replacing the value at an already-present key changes a target reference's count by exactly $[new=t] - [old=t]$ (stated additively to dodge `Nat` subtraction); applying this once for each of the two slots being swapped, the two correction terms cancel exactly, so `swap` is provably an *exchange* of live values rather than a plain overwrite, and the overall `Rld`-count comes out unchanged, not merely bounded.

H3 the region-touching branches' own side conditions keep everything inside one Open region; `swap_corollary_region_loc_eq / _stack_loc_eq` carry pre-existing objects' locations across.

S2, S3 (the two largest proofs in the file) need both the displaced old value and the newly-copied value to respect index bounds simultaneously; `swap_corollary_stack_field_eq_yfRef / _region_field_eq_yfRef` confirm the overwritten field held exactly the value being displaced (a genuine exchange, not a blind overwrite), and existing *S2/S3* facts about both slots' pre-mutation values carry over.

HS1, HS2 both displaced values were already live before the swap, so nothing new enters *refs*.

2.11 allPreserve_ValidConfig

Case-splits on the reified *Stmt* type (`Mutation/Stmt.lean`) and dispatches to the nine theorems above. No independent content.

3 Gc/Reachability/Reachable: semantics and the confinement toolkit

3.1 Semantics-level theorems (`Semantics.lean`)

ReachableStep, *RegionReachable*, *FrameReachable*, *StackReachable*, *FrameRoot* are defined in full in `main.tex`.

RegionReachable_iff_reflTransGen $\text{RegionReachable}(rid, ref) \iff \exists region, H(rid)=region \wedge \text{ReflTransGen}(\rightarrow)(\text{Old}(region.\text{bridgeObjectId})) ref$. Proof: the inductive **bridge/step** constructors correspond exactly to `ReflTransGen`'s `refl/tail`; induction on either side transfers directly.

FrameReachable_iff_reflTransGen Same idea with two possible roots (*FrameRoot*: a variable's value, or the frame's own bridge object) instead of one.

RegionReachable_implies_FrameReachable If frame *F* owns region *rid* (i.e. $F.\text{regionId} = rid$), $\text{RegionReachable}(rid, \cdot) \rightarrow \text{FrameReachable}(F.\text{index}, \cdot)$: everything reachable from a region's own bridge object is, a fortiori, reachable from any frame currently executing in that region. Proof: induction on the *RegionReachable* witness, replacing its **bridge** base case (root at the region's bridge object) with *FrameReachable.bridge* (root at the *frame*'s own region's bridge object, which is the same object since *F* owns *rid*) and carrying every **step** constructor across unchanged. Not currently used elsewhere in the development, but stated as a general connecting fact between the two reachability notions.

3.2 The confinement toolkit (`Lemmas/Common.lean`)

stack_step_index_le *S2* restated at the *ReachableStep* level: a `stack`→`stack` hop's target frame index is $\leq$ the source's. Proof: unfold `objAt?` at the source, apply *S2* directly to the resulting membership fact.

successor_of_region_object *H3* forward, at the *ReachableStep* level: if oid_A is located in region *rid* and $oid_A \rightarrow oid_C$, then oid_C is located in *rid* too. Proof: unfold `objAt?` to get $oid_C \in$

some object's *refs* inside *rid*'s *objMap*, lift that into *rid*'s own *Region.refs* (*mem_region_refs_of_mem_objMap*), then apply *H3* directly. Used by *exit*'s *exit_avoid_popped_step* (§4) to show a chain confined to the popped frame's own region can't accidentally have stepped onto the popped frame's slot.

predecessor_of_region_object Backward confinement, Open region: anything stepping *into* an in-region object is itself in that region or on the stack, never a bare Rld or a different region. *Proof*: case on the predecessor; an Rld source is ruled out by *open_rid_no_step* (same region) or by *H3* forcing the other region's bridge object's own location to disagree; an Old source is reclassified by *H3* applied to whatever it currently resolves into.

predecessor_of_closed_region_object Same statement, Closed region: the "on stack" branch is now impossible (*closed_region_not_owned*, *L2*'s contrapositive), and the Rld branch (the portal) becomes an admissible predecessor instead of a contradiction.

predecessor_of_stack_object Backward confinement, stack: anything stepping into a stack-resident object is itself stack-resident (an Rld or region-object predecessor is ruled out via *H3* forcing a *Rgn* location, contradicting *Stk*).

SafeRef / predecessor_safe / safe_reflTransGen_transport *SafeRef_{rid}(ref)*: in *rid* or on the stack, never a bare Rld. Safety propagates backward one hop (combines the two predecessor lemmas above); if *ReachableStep* agrees on every Old-sourced step between *cfg, cfg'*, any chain ending at a safe target transports wholesale (induction on the chain, applying *predecessor_safe* at each step).

stack_container_confined Anything *FrameReachable*-reaching a stack-resident object at index *fid_C* is rooted from index $\geq fid_C$. *Proof*: restate as *Ref1TransGen*, induct forward from the fixed root generalized over the chain's (growing) end, applying *predecessor_of_stack_object* and *stack_step_index_le* at each hop; the two root shapes close via *S2* directly.

stack_step_region_index_le *S3* restated at the *ReachableStep* level (the region-side analogue of *stack_step_index_le*).

region_container_confined Anything reaching an object inside an *Open* region owned by frame *ownerFrame* is rooted from index $\geq ownerFrame.index$. *Proof*: same shape as *stack_container_confined*, dispatching on *predecessor_of_region_object*'s two surviving cases (stay in-region, or escape to stack via *stack_step_region_index_le* plus *merge_corollary_regionId_unique_index* to convert "same region" into "same index").

FrameReachable_extend If *FrameReachable*(*fid, x*) and $x \xrightarrow{*} y$, then *FrameReachable*(*fid, y*): reachability is closed under continuing a chain further. *Proof*: induction on the *Ref1TransGen* witness for $x \xrightarrow{*} y$, applying the *FrameReachable.step* constructor once per extra hop. Used by *CR6* to glue a portal hop onto an already-established stack-rooted chain.

RegionReachable_of_FrameReachable_closed For a *Closed* region: any *FrameReachable* witness to an in-region object is already a *RegionReachable* one. *Proof*: backward induction over the chain, dispatching on *predecessor_of_closed_region_object* (continue in-region, or stop at the portal hop).

FrameReachable_rid_of_closed_container Companion: reaching anything inside a *Closed* region forces the region's own Rld to already be *FrameReachable* too. Same backward induction, different conclusion at the portal-hop case.

3.3 CR6 (CR6.lean)

StackReachable_iff_RegionReachable_of_closed For *H(rid)* *Closed* and *oid* inside it: *given* *Rld(rid)* is already *StackReachable*, *StackReachable*(*Old(oid)*) $\iff$ *RegionReachable*(*rid, Old(oid)*). *Proof*: forward direction is *RegionReachable_of_FrameReachable_closed* directly (doesn't

even need the hypothesis); backward direction glues the portal hop onto the already-reachable $\text{Rld}(rid)$ via `FrameReachable_extend`.
`not_StackReachable_rid_implies_not_StackReachable_objects_of_closed` If $\text{Rld}(rid)$ is *not* stack-reachable, nothing inside the Closed region is either. *Proof*: contrapositive of `FrameReachable_rid_of_closed_container`.

4 *FrameReachableAtLaterFrame*

Statement (`main.tex`'s *FRALF*): an object in an earlier frame's region, already frame-reachable from a later frame, is already frame-reachable from its own frame. `FrameReachableAtLaterFrame_step` cmd is its single-step-preservation form.

4.1 `RuntimeConfig.start_frameReachableAtLaterFrame`

Vacuous: a single frame makes $F.fid < F'.fid$ unsatisfiable.

4.2 The nine per-operation proofs

Five distinct shapes appear.

(A) Additive, via whole-chain safety transport — enter, exit. Nothing is allocated and no existing content changes (only a status flip). Every chain ending at the tracked object is shown *SafeRef* and transported wholesale.

`frameReachableAtLaterFrame_step_enter` Key lemmas: `enter_oid_step_iff` (step relation agrees on Old-sourced steps), `safe_reflTransGen_transport`, `enter_regionId_ne`, `enter_frame_mem_up/down`. The one wrinkle: if the “later frame” turns out to be the freshly-pushed one, its only possible root is the entered region's bridge object, and a safety argument shows that can't be the (different) tracked object.

`frameReachableAtLaterFrame_step_exit` Key lemmas: `exit_frame_reachable_transport/_backward` (built from `exit_avoid_popped_step`, `exit_oid_step_iff_of_ne_popped`, and `safe_reflTransGen_root_safe`), `exit_regionId_ne`.

(B) Additive, fresh entity is a dead end — makeObjStack, makeObjRegion, makeRegion. Something new is allocated, but any chain trying to route through it collapses to a contradiction (a fresh id has no stored refs and doesn't yet resolve anywhere).

`frameReachableAtLaterFrame_step_makeObjStack` Key lemmas: `makeObjStack_frame_reachable_iff` (pre-existing frames below the last index), `freshObjectId_loc_ne_rgn`, `freshObjectId_objAt_none` (the fresh id's chain is a dead end: it doesn't resolve into the tracked region, and it has no outgoing step either).

`frameReachableAtLaterFrame_step_makeObjRegion` Same shape; key lemmas `makeObjRegion_frame_reachable_iff`, `makeObjRegion_regionId_ne`, plus the same two fresh-id dead-end facts.

`frameReachableAtLaterFrame_step_makeRegion` Same shape, one level richer (a fresh region *and* its fresh bridge object): `makeRegion_frame_reachable_iff`, `makeRegion_reflTransGen_transport_down` (chains avoiding the fresh Rld transport freely), `makeRegion_corollary_loc_fresh`, `freshRegionId_no_step`. A chain hitting the fresh Rld can take at most one more hop (into the equally-fresh, empty bridge object) before dying.

(C) **Root-substitution** — `varAsgn`. `varAsgn_step_eq` shows the *entire* `ReachableStep` relation is literally unchanged (even the bridge-var branch: the touched region stays Open, so no `Rld`-step existed either side to disturb). Only a variable’s *bound value* changes, to something already independently reachable.

`frameReachableAtLaterFrame_step_varAsgn` Key lemmas: `varAsgn_step_eq`, `varAsgn_frame_reachable_iff` (pre-existing frames), and, for the mutated frame’s own new root, `resolveFA_frameReach` (the field value being copied already had an independent `FrameReachable` witness) combined with `region_container_confined` to bound where that witness’s frame can sit relative to the tracked frame.

(D) **Content-mutating, genuine escape induction** — `fieldAsgn`, `swap`. The step relation changes at exactly the mutated container. See the detailed technique note below.

`frameReachableAtLaterFrame_step_fieldAsgn` Key lemmas: `fieldAsgn_step_iff_of_ne` (step relation agrees off the mutated id `oidC`), `fieldAsgn_frame_root_iff`, `region_container_confined/stack_container_confined` (bounding `oidC`’s own owner index), then a `Relation.ReflTransGen.induction_on` walk checking each hop against `fieldAsgn_stack_oidC_step_of_ne_oidY/_region_oidC_step_of_ne_oidY`; a match escapes onto the field’s new value via `resolveV_frameRoot`, and `reflTransGen_region_open_ne_absurd` rules out the region-branch’s new value ever reaching a *different* Open region.

`frameReachableAtLaterFrame_step_swap` Same shape across all four branches; key lemmas `swap_frame_reachable_iff_of_lt` (suspended chains can’t reach the mutated container at all, by the owner-index bound, so no escape machinery is even needed for them), `swap_escape/swap_root_escape/swap_root_escape_rid` (the shared “new value has its own independent root” argument, reused across branches), `swap_stack_yoid_step_of_ne/_region_stack/_bridge_` (per-branch step-agreement off the mutated id), `reflTransGen_rid_source_open_absurd`.

(E) **Reference-deleting** — `merge`. See the walkthrough note below; the key fact is that a live, about-to-be-deleted `Rld(rid’)` can never be a step *target*.

`frameReachableAtLaterFrame_step_merge` Key lemmas: `merge_ridPrime_not_step_target` (*H2*-based: a predecessor would push the count past 1), `merge_chain_transport_down` (any `Rld(rid’)`-avoiding chain transports for free), `merge_ridPrime_reflTransGen_iff` (a chain rooted at `Rld(rid’)` is the same chain, one hop shorter, rooted at its bridge object instead), `merge_frame_reachable_iff`, `merge_regionId_ne_ridPrime`.

Technique note (D). The step relation changes at exactly one container, id `oidC`. `region_container_confined/stack_container_confined` bound `oidC`’s reachable set to frame indices $\geq$ the active (mutating) frame’s own index, `fidactive`. Any chain confined strictly below `fidactive` never touches `oidC` and transports untouched — this covers every strictly-suspended frame automatically. The one remaining case is a chain rooted exactly at the active frame, walked forward hop by hop (`Relation.ReflTransGen.head_induction_on`): at the one hop that could be the newly-written edge, the chain escapes onto the field’s new value, which has its own independent root (`resolveV_frameRoot`), reducing the argument back to the already-handled confined case.

4.3 `frameReachableAtLaterFrame_step_all`

Dispatcher, case-splitting on `Stmt`.

5 *StackReachable_invariant_for_suspended_region_objects*

Statement (`main.tex`): for a suspended frame F and an object oid in F 's region, $StackReachable(Old(oid))$ is unchanged by any single operation, *given* $FRALF$ holds beforehand. Proved as an $\iff$, not a one-way implication.

Each `StackReachableInvariant`/`<Op>.lean` reuses *exactly* the case analysis of its §3-numbered $FRALF$ sibling above (same key lemmas per operation), applied in both directions: “reachable before $\Rightarrow$ reachable after” is close to a direct instance of the $FRALF$ -style transport; “reachable after $\Rightarrow$ reachable before” is where the escape argument (shapes D/E) is actually exercised, since the newly-written edge or the about-to-be-deleted reference exists on the $cfg' \rightarrow cfg$ side, not the other.

`stackReachable_invariant_enter` Both directions via `safe_reflTransGen_transport` (called with the step-iff in each direction).

`stackReachable_invariant_exit` Both directions via `exit_reflTransGen_transport/_backward`.

`stackReachable_invariant_makeObjStack` / `makeObjRegion` / `makeRegion` Both directions via the corresponding `<op>_frame_reachable_iff`, with the fresh-id dead-end lemmas closing the one new case per direction.

`stackReachable_invariant_varAsgn` Both directions via `varAsgn_step_eq + varAsgn_frame_reachable_iff`; the root-substitution argument (`resolveFA_frameReach`) is needed in the forward direction here (a suspended-frame witness reaching the pre-mutation root must still resolve after the var is rebound).

`stackReachable_invariant_fieldAsgn` Forward direction: confined transport suffices (the mutated id's owner-index bound already excludes it from any suspended-frame chain). Backward direction: the full escape argument (shape D), since the new edge exists only in cfg' .

`stackReachable_invariant_swap` Same asymmetry as `fieldAsgn`, across all four branches.

`stackReachable_invariant_merge` Forward: confined transport via `merge_chain_transport_down`. Backward: the `merge_ridPrime_reflTransGen_iff` substitution, since the deleted reference's replacement only exists in cfg' .

5.1 `stackReachable_invariant_all`

Dispatcher, case-splitting on *Stmt*.

6 What's not proved yet

No `Guarantees.lean`-style top-level GC-safety theorem sits on top of *StackReachable_invariant_for_suspended_region_objects* yet — nothing in this codebase currently concludes “therefore the collector never frees something still reachable.” Every theorem in §4–5 is also a *single-step* fact, quantified over one operation at a time; composing it over an arbitrary-length trace of operations (the literal report.pdf claim, whose own qualifier is invariance for as long as a region *stays* suspended, not just across the one operation that happens to run while it's suspended) would need an induction over trace length on top of what's here, and is not currently being pursued.